

Job-Finder Codebase - Important Code Documentation

Generated: July 1, 2026

Table of Contents

- 1. [Backend Server Setup](#)
- 2. [Authentication](#)
- 3. [Data Models](#)
- 4. [Job Management](#)
- 5. [AI Resume Analysis](#)
- 6. [Frontend Application](#)

Backend Server Setup

Entry Point: backend/src/index.ts

```
import 'dotenv/config';
import express, { Express } from 'express';
import cors from 'cors';
import morgan from 'morgan';
import compression from 'compression';
import { connectDB } from './config/database';

// Import routes
import authRoutes from './routes/authRoutes';
import jobRoutes from './routes/jobRoutes';
import resumeRoutes from './routes/resumeRoutes';
import adminRoutes from './routes/adminRoutes';
import resumeAnalyzerRoutes from './routes/resumeAnalyzerRoutes';

const app: Express = express();
const PORT = process.env.PORT || 5000;
const normalizeOrigin = (value: string) => value.trim().replace(/\/+$/, '');

const configuredOrigins = (process.env.FRONTEND_URL || '')
  .split(',')
  .map((entry) => entry.trim())
  .filter(Boolean)
  .map(normalizeOrigin);

// Middleware
app.use(
  cors({
    origin: (origin, callback) => {
      if (!origin) {
        callback(null, true);
        return;
      }

      const normalizedRequestOrigin = normalizeOrigin(origin);

      if (configuredOrigins.includes(normalizedRequestOrigin)) {
        callback(null, true);
        return;
      }

      const localOriginPattern = /^https?:\/\/(localhost|127\.0\.0\.1|d{1,3}(\?:\d{1,3}){3})(:\d+)?$/;
      if (localOriginPattern.test(origin)) {
        callback(null, true);
        return;
      }

      callback(new Error('Not allowed by CORS'));
    },
  })
);
app.use(compression()); // Enable gzip compression for all responses
app.use(morgan('dev'));
app.use(express.json({ limit: '50mb' }));
app.use(express.urlencoded({ limit: '50mb', extended: true }));

// Static files for uploads
app.use('/uploads', express.static('uploads'));

// Connect to database
connectDB();

/**
 * Routes
 */
app.use('/api/auth', authRoutes);
```

```
app.use('/api/jobs', jobRoutes);
app.use('/api/resume', resumeRoutes);
app.use('/api/admin', adminRoutes);
app.use('/api/resume-analyzer', resumeAnalyzerRoutes);

/**
 * Health check
 */
app.get('/api/health', (req, res) => {
  res.status(200).json({ message: 'Server is running', timestamp: new Date() });
});

/**
 * Error handling middleware
 */
app.use((err: any, req: express.Request, res: express.Response, next: express.NextFunction) => {
  console.error('Error:', err);
  res.status(err.status || 500).json({
    message: err.message || 'Internal server error',
  });
});

// Start server
app.listen(PORT, () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`);
  console.log(`🌐 Environment: ${process.env.NODE_ENV}`);
});

export default app;
```

Key Features:

- CORS configuration with origin normalization
- Gzip compression for response optimization
- Morgan logging middleware
- File upload support (50mb limit)
- Multiple route handlers
- Health check endpoint
- Centralized error handling

Authentication

Middleware: backend/src/middleware/auth.ts

```

import jwt from 'jsonwebtoken';
import { Request, Response, NextFunction } from 'express';

export interface AuthRequest extends Request {
  user?: {
    id: string;
    email: string;
    role: string;
  };
}

/**
 * Authentication middleware - verify JWT token
 */
export const authMiddleware = (
  req: AuthRequest,
  res: Response,
  next: NextFunction
): void => {
  try {
    const token = req.headers.authorization?.split(' ')[1];

    if (!token) {
      res.status(401).json({ message: 'No token provided' });
      return;
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret') as {
      id: string;
      email: string;
      role: string;
    };

    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ message: 'Invalid token' });
  }
};

/**
 * Role-based access control middleware
 */
export const roleMiddleware = (allowedRoles: string[]) => {
  return (req: AuthRequest, res: Response, next: NextFunction): void => {
    if (!req.user) {
      res.status(401).json({ message: 'Not authenticated' });
      return;
    }

    if (!allowedRoles.includes(req.user.role)) {
      res.status(403).json({ message: 'Insufficient permissions' });
      return;
    }

    next();
  };
};

```

Controller: backend/src/controllers/authController.ts

```

import { Response } from 'express';
import { AuthRequest } from '../middleware/auth';
import User from '../models/User';
import { hashPassword, comparePassword, generateToken } from '../utils/auth';
import { admin, isFirebaseInitialized } from '../config/firebase';

const isValidLoginRole = (role: unknown): role is 'user' | 'company' => {
  return role === 'user' || role === 'company';
};

const formatUserResponse = (user: any) => {
  const userObject = user.toObject ? user.toObject() : user;
  const { password, __v, ...safeUser } = userObject;
  return safeUser;
};

/**
 * Firebase Register - Create user with Firebase ID token
 */
export const firebaseRegister = async (req: AuthRequest, res: Response): Promise<void> => {
  try {
    const { idToken, email, name, role } = req.body;

    if (!idToken || !email || !name) {
      res.status(400).json({ message: 'ID token, email, and name required' });
    }
  }
};

```

```

    return;
}

if (!isFirebaseInitialized) {
  res.status(500).json({ message: 'Firebase not configured' });
  return;
}

// Verify Firebase ID token
try {
  await admin.auth().verifyIdToken(idToken);
} catch (error: any) {
  console.error('Token verification error:', error.message);
  res.status(401).json({ message: 'Invalid or expired token' });
  return;
}

const normalizedEmail = email.toLowerCase();

// Check if user already exists
let user = await User.findOne({ email: normalizedEmail });

if (!user) {
  // Create new user
  user = await User.create({
    name,
    email: normalizedEmail,
    role: role || 'user',
  });
  console.log('New Firebase user created:', normalizedEmail);
} else {
  console.log('User already exists:', normalizedEmail);
}

// Generate token
const token = generateToken(user._id.toString(), user.email, user.role);

res.status(201).json({
  message: 'User registered successfully',
  token,
  user: formatUserResponse(user),
});
} catch (error) {
  console.error('Firebase registration error:', error);
  res.status(500).json({ message: 'Registration failed' });
}
};

/**
 * Firebase Login - Login with Firebase ID token
 */
export const firebaseLogin = async (req: AuthRequest, res: Response): Promise<void> => {
  try {
    const { idToken, email, role } = req.body;

    if (!idToken || !email) {
      res.status(400).json({ message: 'ID token and email required' });
      return;
    }

    if (!isFirebaseInitialized) {
      res.status(500).json({ message: 'Firebase not configured' });
      return;
    }

    // Verify Firebase ID token
    try {
      await admin.auth().verifyIdToken(idToken);
    } catch (error: any) {
      console.error('Token verification error:', error.message);
      res.status(401).json({ message: 'Invalid or expired token' });
      return;
    }

    const normalizedEmail = email.toLowerCase();

    // Find user
    const user = await User.findOne({ email: normalizedEmail });

    if (!user) {
      res.status(404).json({ message: 'This email is not registered. Please sign up.' });
      return;
    }

    if (isValidLoginRole(role) && user.role !== role && user.role !== 'admin') {
      user.role = role;
    }
  }
};

```

```

    await user.save();
  }

  // Generate token
  const token = generateToken(user._id.toString(), user.email, user.role);

  res.status(200).json({
    message: 'Login successful',
    token,
    user: formatUserResponse(user),
  });
} catch (error) {
  console.error('Firebase login error:', error);
  res.status(500).json({ message: 'Login failed' });
}
};

```

Authentication Flow:

- Firebase ID token verification
- JWT token generation for authenticated sessions
- Role-based access control (user, company, admin)
- Email normalization and validation

Data Models

User Model: backend/src/models/User.ts

```

import { Schema, model, Document } from 'mongoose';

export interface IUser extends Document {
  _id: string;
  name: string;
  email: string;
  password?: string;
  role: 'user' | 'company' | 'admin';
  phone?: string;
  avatar?: string;
  resume?: string;
  resumeUrl?: string;
  resumeAnalysis?: {
    score: number;
    strengths: string[];
    weaknesses: string[];
    extractedText: string;
    analyzedAt: Date;
  };
  bio?: string;
  location?: string;
  skills?: string[];
  experience?: number;
  isVerified: boolean;
  createdAt: Date;
  updatedAt: Date;
}

const userSchema = new Schema<IUser>(
  {
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true, lowercase: true },
    password: { type: String, required: false },
    role: {
      type: String,
      enum: ['user', 'company', 'admin'],
      default: 'user',
    },
    phone: String,
    avatar: String,
    resume: String,
    resumeUrl: String,
    resumeAnalysis: {
      score: Number,
      strengths: [String],
      weaknesses: [String],
      extractedText: String,
      analyzedAt: Date,
    },
  },
  {
    bio: String,
  }
);

```

Job Model: backend/src/models/Job.ts

```
import { Schema, model, Document, Types } from 'mongoose';

export interface IJob extends Document {
  _id: string;
  title: string;
  description: string;
  requirements: string[];
  skills: string[];
  salary?: {
    min: number;
    max: number;
    currency: string;
  };
  jobType: 'full-time' | 'part-time' | 'contract' | 'internship';
  experienceLevel: 'entry' | 'mid' | 'senior' | 'lead';
  location: string;
  workplaceType: 'remote' | 'on-site' | 'hybrid';
  company: string;
  companyId: Types.ObjectId;
  applicationDeadline: Date;
  applicants: Types.ObjectId[];
  status: 'published' | 'draft' | 'closed' | 'rejected';
  approvedBy?: Types.ObjectId;
  rejectionReason?: string;
  views: number;
  featured: boolean;
  createdAt: Date;
  updatedAt: Date;
}

const jobSchema = new Schema<IJob>(
  {
    title: { type: String, required: true },
    description: { type: String, required: true },
    requirements: [String],
    skills: [String],
    salary: {
      min: Number,
      max: Number,
      currency: { type: String, default: 'INR' },
    },
    jobType: {
      type: String,
      enum: ['full-time', 'part-time', 'contract', 'internship'],
      default: 'full-time',
    },
    experienceLevel: {
      type: String,
      enum: ['entry', 'mid', 'senior', 'lead'],
      default: 'mid',
    },
  },
  {
    timestamps: true,
  },
);

const Job = model<IJob>('Job', jobSchema);
```

Data Structure:

- User roles: user, company, admin
- Job types: full-time, part-time, contract, internship
- Experience levels: entry, mid, senior, lead
- Workplace types: remote, on-site, hybrid
- Resume analysis scoring and tracking

Job Management

Job Controller: backend/src/controllers/jobController.ts

```
import { Response } from 'express';
import { AuthRequest } from '../middleware/auth';
import Job from '../models/Job';
import Application from '../models/Application';
import { analyzeResume, predictHireLikelihood, generateInterviewQuestions } from '../services/aiResumeAnalysis';
import User from '../models/User';
import axios from 'axios';
import fs from 'fs';
import os from 'os';
import path from 'path';
import { extractResumeText } from '../utils/resumeParser';

const extractResumeTextFromUrl = async (resumeUrl: string): Promise<string> => {
  try {
    const response = await axios.get(resumeUrl, { responseType: 'arraybuffer' });
    const buffer = Buffer.from(response.data);
    const parsedUrl = new URL(resumeUrl);
    const extFromUrl = path.extname(parsedUrl.pathname).toLowerCase();
    const ext = extFromUrl === '.doc' ? '.docx' : extFromUrl || '.pdf';
    const tempPath = path.join(os.tmpdir(), `resume-${Date.now()}${ext}`);

    fs.writeFileSync(tempPath, buffer);

    return extractResumeText(tempPath);
  } catch (error) {
    console.error('Error extracting resume text from URL:', error);
    return '';
  }
};
```

```

    const text = await extractResumeText(tempPath);
    fs.unlinkSync(tempPath);
    return text;
  } catch (error) {
    console.warn('⚠️ Failed to extract resume text from URL:', error);
    return '';
  }
};

const getUserResumeContext = async (userId: string) => {
  const user = await User.findById(userId).select('resume resumeUrl resumeAnalysis');
  const resumeToUse = user?.resumeUrl || user?.resume;

  if (!resumeToUse) {
    throw new Error('Please upload your resume in profile before checking AI matching');
  }

  let resumeText = user?.resumeAnalysis?.extractedText || '';
  if (!resumeText && typeof resumeToUse === 'string' && resumeToUse.startsWith('http')) {
    resumeText = await extractResumeTextFromUrl(resumeToUse);
  }

  return {
    resumeToUse,
    resumeText: resumeText || '',
  };
};

const buildUserPredictionForJob = async (userId: string, job: any) => {
  const { resumeText, resumeToUse } = await getUserResumeContext(userId);

  const prediction = await predictHireLikelihood(resumeText, {
    title: job.title,
    description: job.description,
    skills: job.skills,
    requirements: job.requirements,
  });

  return { prediction, resumeToUse };
};

/**
 * Create a new job posting
 */
export const createJob = async (req: AuthRequest, res: Response): Promise<void> => {
  try {
    if (!req.user) {
      res.status(401).json({ message: 'Not authenticated' });
      return;
    }

    if (req.user.role !== 'company' && req.user.role !== 'admin') {
      res.status(403).json({ message: 'Only companies and admins can post jobs' });
      return;
    }

    const {
      title,
      description,
      requirements,
      skills,
      salary,
      jobType,
      experienceLevel,
      location,
      workplaceType,
      applicationDeadline,
      company,
    } = req.body;

    const job = await Job.create({
      title,
      description,
      requirements,
      skills,
      salary,
      jobType,
      experienceLevel,
      location,
      workplaceType,
      company,
      companyId: req.user.id,
      applicationDeadline,
      status: 'published',
    });
  }
};

```

```

    res.status(201).json({
      message: 'Job created successfully',
      job,
    });
  } catch (error) {
    console.error('Create job error:', error);
    res.status(500).json({ message: 'Failed to create job' });
  }
};

/**
 * Get all jobs with filters and search
 */
export const getJobs = async (req: AuthRequest, res: Response): Promise<void> => {
  try {
    const { search, location, jobType, experienceLevel, workplaceType, page = 1, limit = 10 } = req.query;

    // Build filter
    const filter: any = { status: 'published' };

    if (search) {
      filter.$or = [
        { title: { $regex: search, $options: 'i' } },
        { description: { $regex: search, $options: 'i' } },
        { company: { $regex: search, $options: 'i' } },
      ];
    }

    if (location) filter.location = { $regex: location, $options: 'i' };
    if (jobType) filter.jobType = jobType;
    if (experienceLevel) filter.experienceLevel = experienceLevel;
    if (workplaceType) filter.workplaceType = workplaceType;

    // Pagination
    const skip = (Number(page) - 1) * Number(limit);

    const jobs = await Job.find(filter)
      .populate('companyId', 'name avatar')
      .skip(skip)
      .limit(Number(limit))
      .sort({ createdAt: -1 });
  }
};

```

Job Management Features:

- Job creation with validation
- Advanced filtering (search, location, type, level, workplace)
- Pagination support
- Resume extraction from URLs
- AI-powered job matching
- Interview question generation

AI Resume Analysis

Service: backend/src/services/aiResumeAnalysis.ts


```

import { GoogleGenerativeAI } from '@google/generative-ai';

/**
 * AI Resume Analysis Service
 * This is a mock implementation. Replace with actual OpenAI API calls
 */

export interface ResumeAnalysis {
  fitScore: number;
  strengths: string[];
  weaknesses: string[];
  missingSkills: string[];
  summary: string;
}

export interface DetailedResumeAnalysis {
  overallScore: number;
  extractedSkills: string[];
  missingSkills: string[];
  suggestions: string[];
  strengthAreas: {
    technical: number;
    experience: number;
    education: number;
    softSkills: number;
  };
  detailedFeedback: {
    formatting: string;
    content: string;
    keywords: string;
    impact: string;
  };
}

export interface HirePredictionResult {
  hireProbability: number;
  verdict: 'likely' | 'uncertain' | 'unlikely';
  missingSkills: string[];
  strengths: string[];
  improvementAreas: string[];
  summary: string;
}

export interface InterviewQuestion {
  id: number;
  question: string;
  category: string;
  difficulty: 'beginner' | 'intermediate' | 'advanced';
}

export interface InterviewQuestionsResult {
  questions: InterviewQuestion[];
  jobTitle: string;
  keySkills: string[];
  generatedAt: string;
}

const DEFAULT_GEMINI_MODEL = process.env.GEMINI_MODEL || 'gemini-2.5-flash';
const RESOLVED_GEMINI_MODEL = DEFAULT_GEMINI_MODEL.replace(/^models\/i, '');

const clampPercentage = (value: unknown, fallback = 50): number => {
  const numericValue = typeof value === 'number' ? value : Number(value);
  if (!Number.isFinite(numericValue)) return fallback;
  if (numericValue < 0) return 0;
  if (numericValue > 100) return 100;
  return Math.round(numericValue);
};

const normalizeStringList = (value: unknown, fallback: string[] = []): string[] => {
  if (!Array.isArray(value)) return fallback;

  return value
    .map(item => (typeof item === 'string' ? item.trim() : String(item || '').trim()))
    .filter(item => item.length > 0)
    .slice(0, 20);
};

const parseJsonFromModelOutput = (text: string): any => {
  const cleanedText = text.replace(/``json|``/gi, '').trim();

```

AI Features:

- Google Generative AI integration (Gemini)
- Resume analysis and scoring
- Skill extraction and matching
- Hire probability prediction
- Interview question generation

- JSON response parsing
-

Frontend Application

Main App Component: `frontend/src/App.tsx`

```

import React, { useEffect } from 'react';
import { BrowserRouter, Routes, Route, useLocation } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import { Navbar } from '../components/Navbar';
import { Footer } from '../components/Footer';
import { ChatbotWidget } from '../components/ChatbotWidget';

// Pages
import { Landing } from '../pages/Landing';
import { Login } from '../pages/Login';
import { Register } from '../pages/Register';
import { JobListings } from '../pages/JobListings';
import { JobDetails } from '../pages/JobDetails';
import { Profile } from '../pages/Profile';
import { AdminDashboard } from '../pages/AdminDashboard';
import { CompanyDashboard } from '../pages/CompanyDashboard';
import { ResumeAnalyzer } from '../pages/ResumeAnalyzer';
import { ProtectedRoute } from '../components/ProtectedRoute';

const ScrollToTop: React.FC = () => {
  const { pathname } = useLocation();

  useEffect(() => {
    window.scrollTo({ top: 0, left: 0, behavior: 'auto' });
  }, [pathname]);

  return null;
};

function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <ScrollToTop />
        <Navbar />
        <Routes>
          { /* Public Routes */ }
          <Route path="/" element={<Landing />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route path="/jobs" element={<JobListings />} />
          <Route path="/jobs/:id" element={<JobDetails />} />
          <Route path="/resume-analyzer" element={<ResumeAnalyzer />} />

          { /* Protected Routes */ }
          <Route
            path="/profile"
            element={
              <ProtectedRoute>
                <Profile />
              </ProtectedRoute>
            }
          />

          <Route
            path="/dashboard"
            element={
              <ProtectedRoute requiredRole={['company', 'admin']}>
                <CompanyDashboard />
              </ProtectedRoute>
            }
          />

          <Route
            path="/admin"
            element={
              <ProtectedRoute requiredRole='admin'>
                <AdminDashboard />
              </ProtectedRoute>
            }
          />

          { /* 404 */ }
          <Route path="*" element={<div className='min-h-screen flex items-center justify-center'><p>Page not found</p></div>} />
        </Routes>
        <Footer />
        <ChatbotWidget />
      </AuthProvider>
    </BrowserRouter>
  );
}

export default App;

```

Frontend Architecture:

- React Router for navigation

- Authentication context provider
- Protected routes with role-based access
- Public and private page routes
- Responsive layout with Navbar and Footer
- Integrated chatbot widget

Routes:

- **Public:** Home, Login, Register, Jobs, Job Details, Resume Analyzer
- **Protected (User):** Profile
- **Protected (Company/Admin):** Dashboard
- **Protected (Admin):** Admin Dashboard

Project Structure Summary

Job-Finder/	
├── backend/	
│ ├── src/	
│ │ ├── index.ts	# Express server entry point
│ │ ├── config/	# Configuration files
│ │ ├── controllers/	# Route handlers
│ │ ├── middleware/	# Auth & RBAC middleware
│ │ ├── models/	# MongoDB schemas
│ │ ├── routes/	# API routes
│ │ ├── services/	# Business logic (AI analysis)
│ │ └── utils/	# Helper functions
│ └── scripts/	# Utility scripts
├── frontend/	
│ └── src/	
│ ├── App.tsx	# Main application component
│ ├── components/	# Reusable React components
│ ├── config/	# Firebase & API config
│ ├── context/	# React context (Auth)
│ ├── hooks/	# Custom hooks
│ ├── pages/	# Page components
│ ├── services/	# API services
│ ├── types/	# TypeScript definitions
│ └── utils/	# Utility functions
└── [Documentation files]	

Key Technologies

Backend

- **Runtime:** Node.js + Express.js
- **Language:** TypeScript
- **Database:** MongoDB + Mongoose
- **Authentication:** Firebase Auth + JWT
- **AI:** Google Generative AI (Gemini)
- **File Upload:** Cloudinary
- **Compression:** gzip

Frontend

- **Framework:** React 18
- **Language:** TypeScript
- **Styling:** Tailwind CSS
- **Build Tool:** Vite
- **Routing:** React Router v6
- **State Management:** React Context API

API Endpoints

Authentication

- POST /api/auth/register - **User registration**
- POST /api/auth/firebase-register - **Firebase registration**
- POST /api/auth/login - **User login**
- POST /api/auth/firebase-login - **Firebase login**

Jobs

- GET /api/jobs - **List jobs with filters**
- POST /api/jobs - **Create job (company/admin)**
- GET /api/jobs/:id - **Job details**
- POST /api/jobs/:id/apply - **Apply to job**

Resume

- POST /api/resume/upload - **Upload resume**
- GET /api/resume - **Get resume**
- POST /api/resume/analyze - **Analyze resume**

Resume Analyzer

- POST /api/resume-analyzer/check-fit - Check job fit
- POST /api/resume-analyzer/interview-questions - Generate questions

Admin

- GET /api/admin/jobs - List all jobs (pending)
 - POST /api/admin/jobs/:id/approve - Approve job
 - POST /api/admin/jobs/:id/reject - Reject job
-

Authentication Flow

1. **Firestore Sign-In** → User authenticates with Google
 2. **ID Token Verification** → Backend verifies Firebase token
 3. **User Creation/Lookup** → MongoDB user record created/retrieved
 4. **JWT Generation** → Backend generates JWT token
 5. **Token Storage** → Frontend stores token in localStorage
 6. **Protected Requests** → JWT included in Authorization header
-

Generated:** July 1, 2026

Document Version: 1.0